Introduction to UML (Unified Modeling Language)

Minsoo Ryu

Hanyang University

msryu@hanyang.ac.kr

Contents

1. Overview of UML
2. Structural Diagrams
3. Behavioral Diagrams

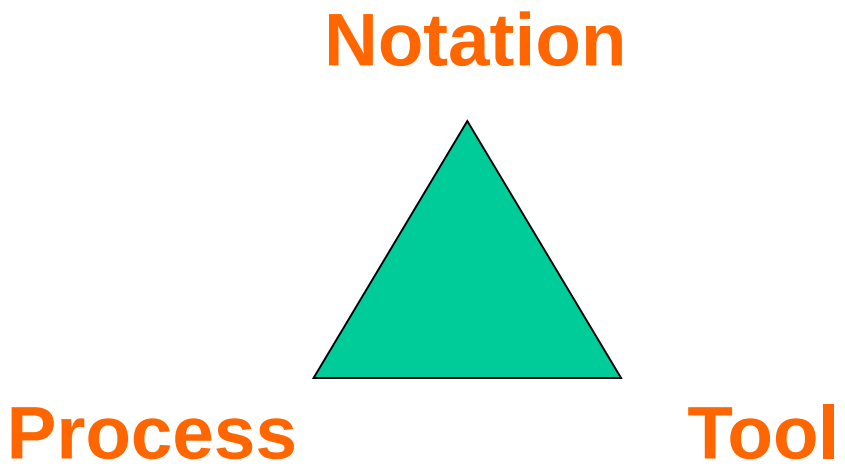
UML Definition

- UML is a visual modeling language for distributed and concurrent systems
  - Visual modeling is the graphic representation of real-world ideas
  - Model is an abstraction that portrays the essentials of a complex problem or structure by filtering out nonessential details, thus making the problem easier to understand

- UML is officially defined at the Object Management Group (OMG) by the UML metamodel, a Meta-Object Facility metamodel (MOF)
  - UML was designed to specify, visualize, construct, and document software-intensive systems

Three Means for Modelling

- Notation: a language for communication (semantics and a form)
- Process: a way of using the notation
- Tool: support to document the artifacts of the work



UML History

- **"Method wars" during the nineties: confusion brought by different notations**
  - **OMT (Object Modeling Technique, James Rumbaugh)**
    - strong in analysis and weaker in design
  - **OOD (Object Oriented Design, Grady Booch)**
    - weaker in analysis and strong in design
  - **OOSE (Object Oriented Software Engineering, Ivar Jacobson)**
    - strong in behavior analysis and weaker in others

- **The end of the method as far as notation is concerned**
  - **Version 0.8 (October, 1995): The first public draft**
  - **Version 1.0 (Jan, 1997)**
    - Proposed by an international consortium called the UML Partners
  - **Version 1.1 (November, 1997)**
    - Semantics were finalized

UML History

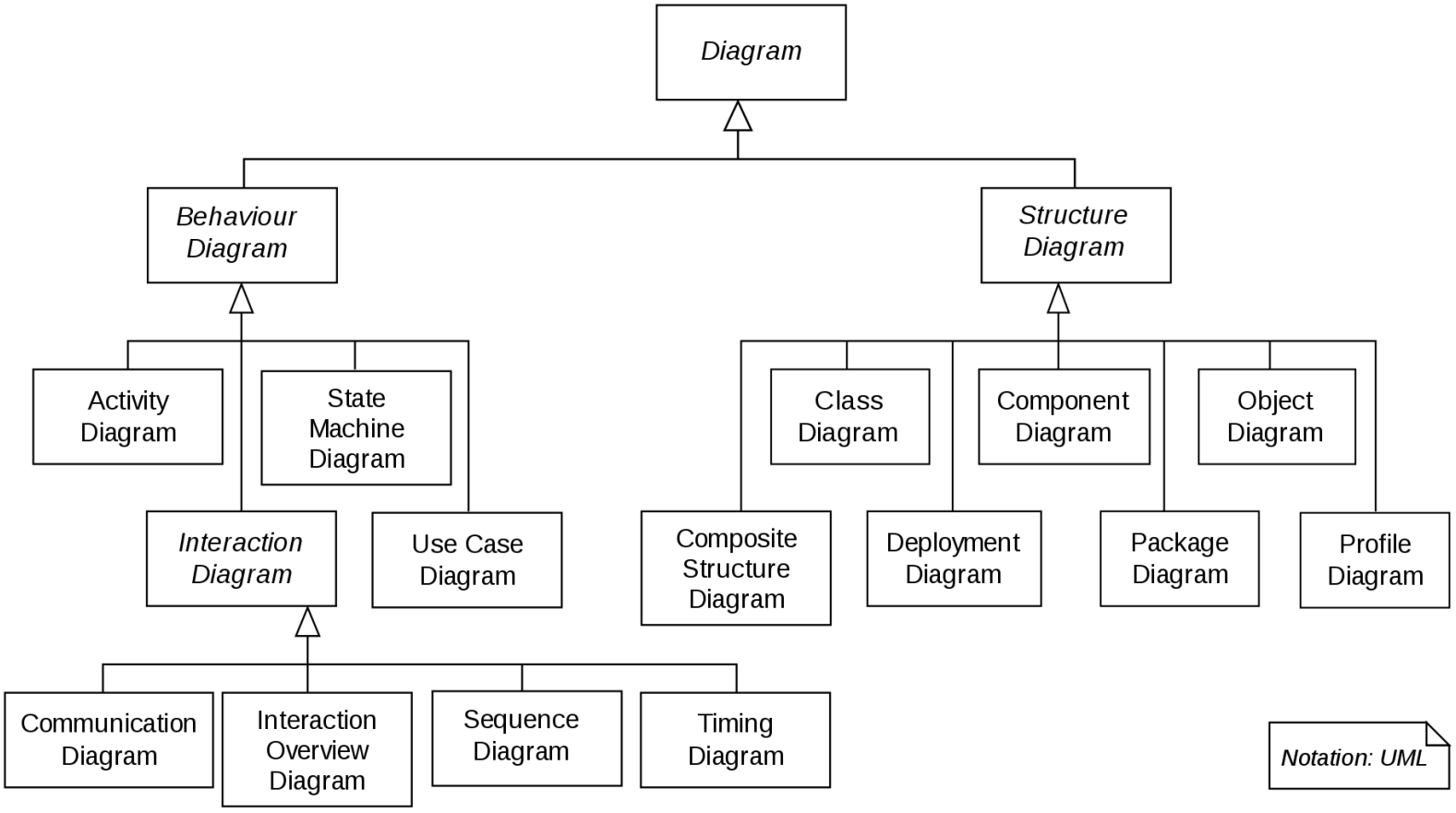
- Version 2.0
  - Several minor revisions: UML 1.3, 1.4, 1.5
  - The first part of UML 2.0, the superstructure which describes the new diagrams and modeling elements available, was adopted by the OMG in October 2004
  - Other parts of UML 2, notably the infrastructure, the Object Constraint Language (OCL) and the diagram interchange were yet to be completed and ratified as of November 2005

- Now UML version 2.5.1 is the latest one (adopted in December 2017)

UML Diagram Classification

- **Structure Diagrams**
  - Emphasize what things must be in the system being modeled:
- **Behavior Diagrams**
  - Emphasize what must happen in the system being modeled
- **Interaction Diagrams (a subset of behavior diagrams)**
  - Emphasize the flow of control and data among the things in the system being modeled

Hierarchy of UML Diagrams



Contents

1. Introduction to UML
2. Structural Diagrams
   - Class Diagram
   - Component Diagram
3. Behavioral Diagrams

Class and Class Diagram

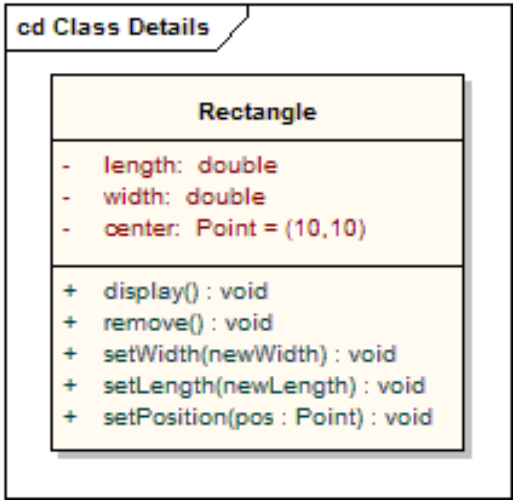
- What is a class?
  - A class defines what information an object can hold and what behavior it can have
  - A class provides a blueprint for an object, and simply provides a template for objects

- A class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, and the relationships between the classes

Class Structure and Representation

- **Attributes** can appear in an optional compartment
  - Each attribute is shown with at least its name, and optionally with its type, initial value, and other properties

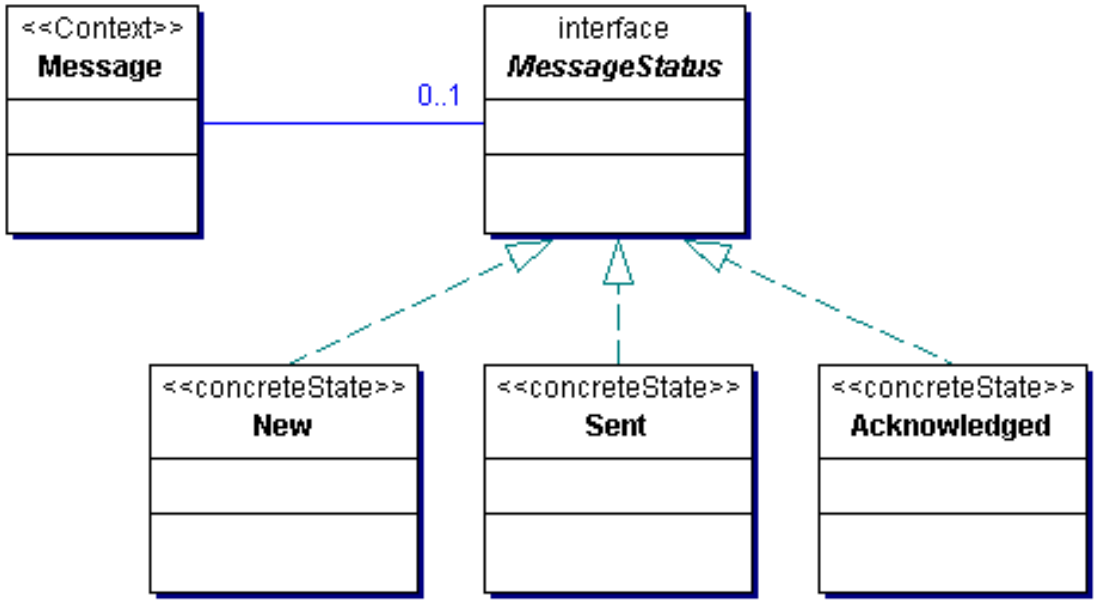
"+" for public
"#" for protected
"−" for private
"~" for package



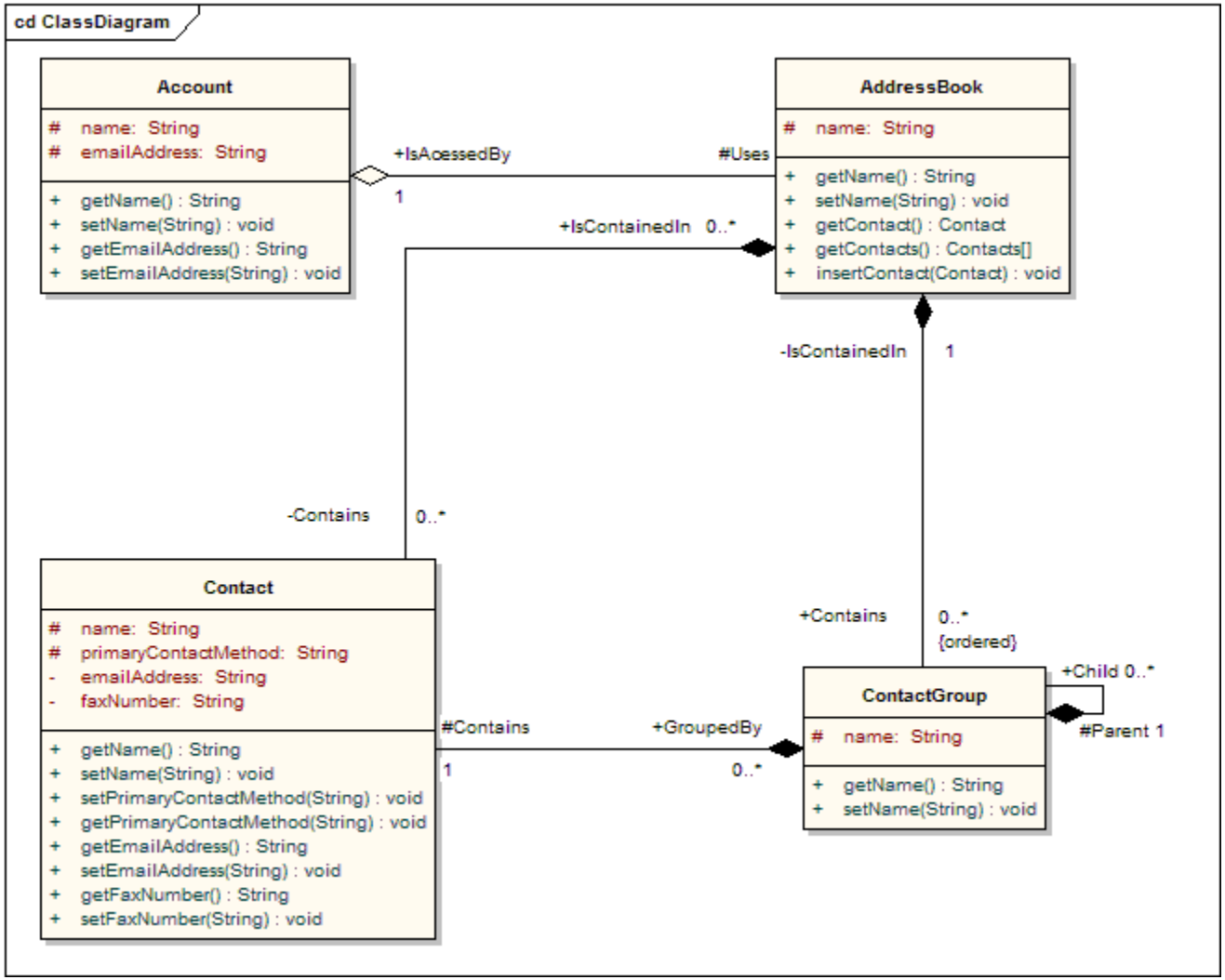
- **Operations** can appear in another compartment
  - Each operation is shown with at least its name, and optionally also with its parameters and return type

Class Stereotypes

- A stereotype conveys a high level category, type, or meaning of an item
  - A stereotype is a mechanism you can use to categorize your classes



Class Diagram



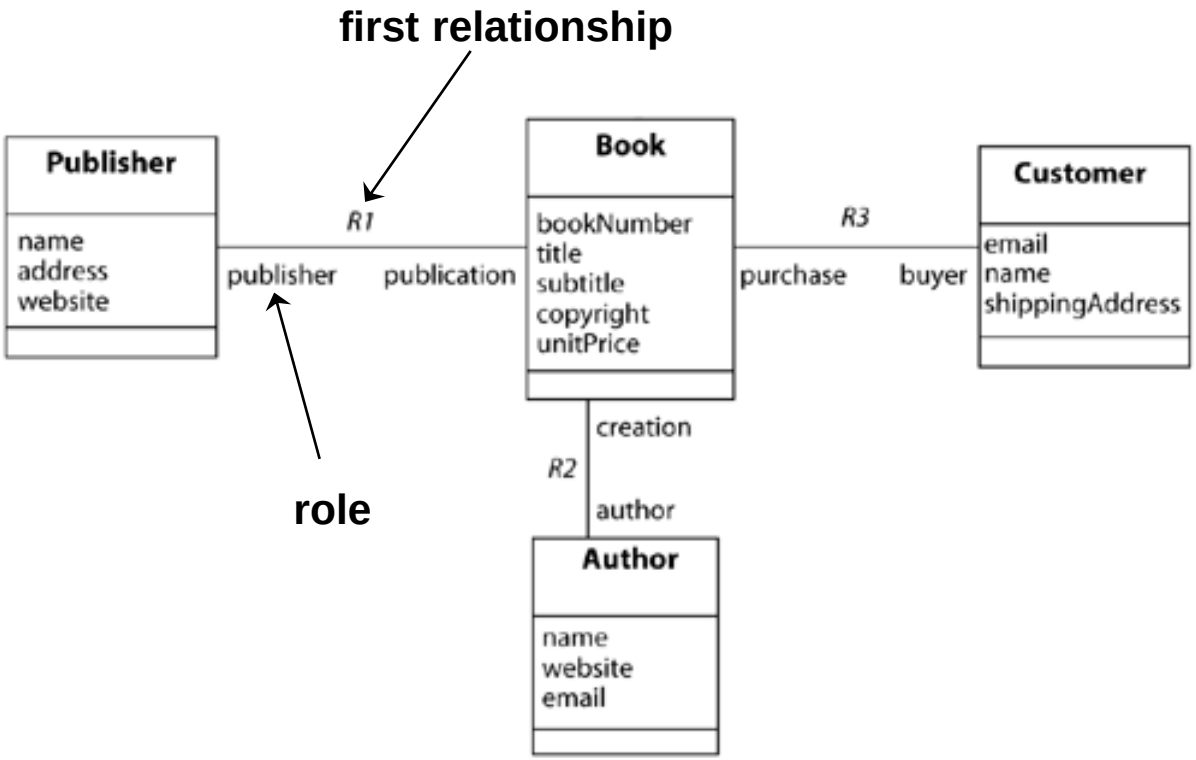
Relationships

- A relationship is a semantic connection between classes
  - It allows one class to know about the attributes, operations, and relationships of another class

- Types of relationships
  - Associations
  - Aggregations
  - Compositions
  - Generalizations
  - Realizations
  - Dependencies

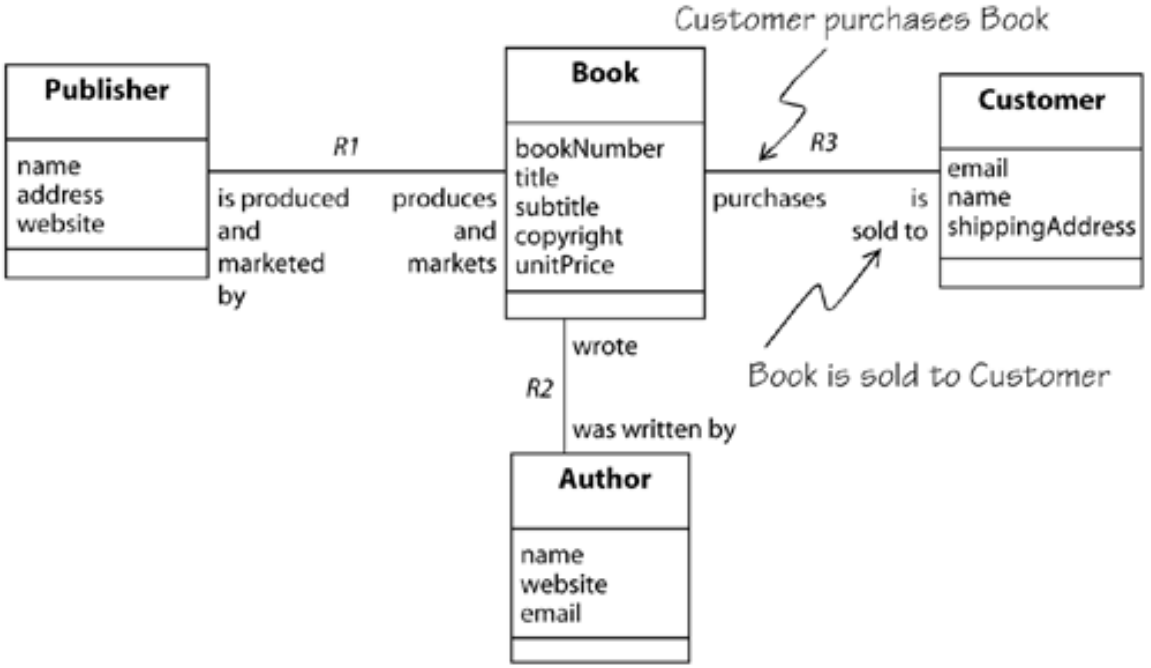
Associations

- When an association connects two classes, it means that the classes communicate (interact) with each other



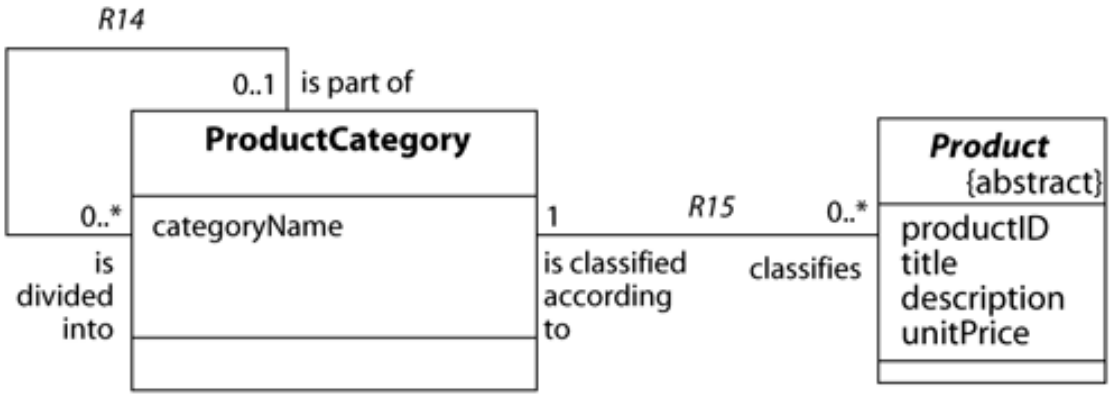
Associations

- Instead of roles, another way to state the association is to write its meaning in terms of verb phrases
  - By convention, the phrase is placed next to the direct object of the sentence



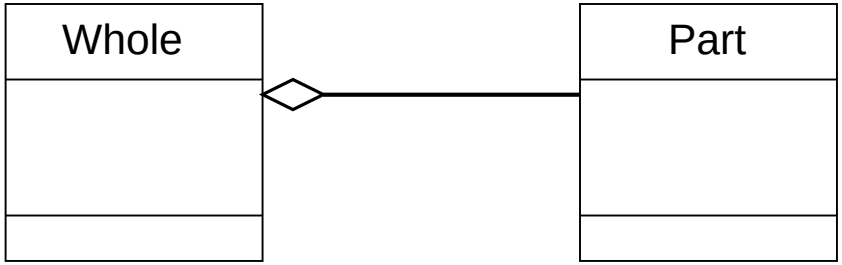
Associations

- Associations can also be reflexive
  - A reflexive association suggests that one instance of a class is related to other instances of the same class



Aggregations

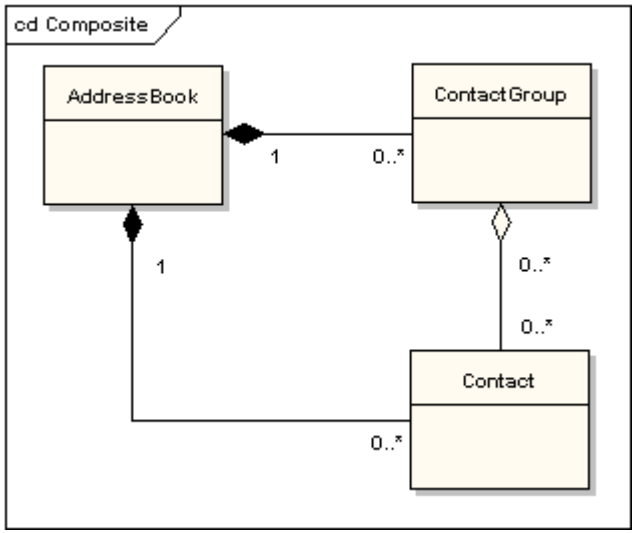
- Aggregations are used to depict elements which are made up of smaller components
  - An "has a" relationship between a whole and its parts



- Aggregation can occur when the contained classes do not have a strong life cycle dependency on the container--essentially, if the container is destroyed, its contents are not

Compositions

- Composition is a stronger variant of the "has a" or association relationship; composition is more specific than aggregation

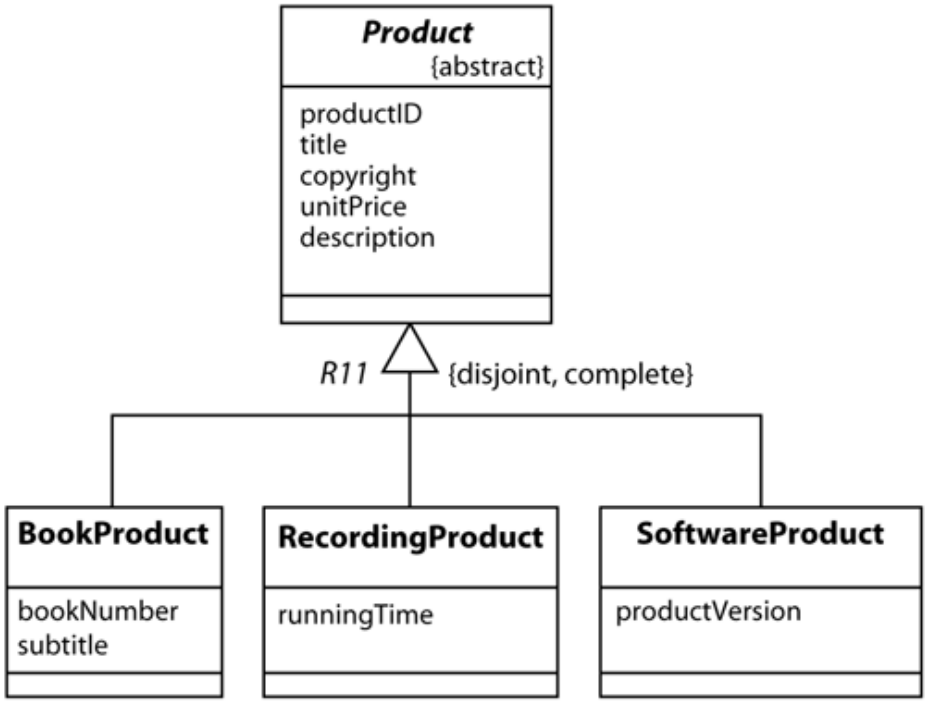


- Composition is a strong life cycle dependency between instances of the container class and instances of the contained class(es)
  - If the container is destroyed, any containees are also destroyed

Generalizations

- The generalization relationship indicates that one of the two related classes (the supertype) is considered to be a more general form of the other (the subtype)
  - "is a" relationship

- Generalizations are used to show an inheritance relationship between two modeling elements (actors, use cases, classes, or packages)

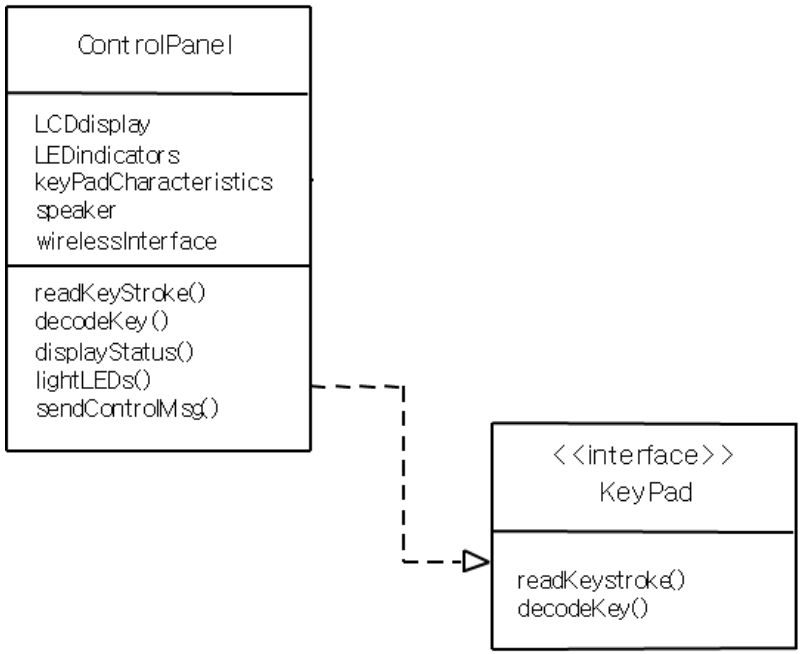


Realizations

- A realization relationship is a relationship between two model elements, in which one model element (the client) realizes the behavior that the other model element (the supplier) specifies
  - Realizes relationships are used to show the relationship between a class and its interface, between a package and its interface, between a component and its interface, or between a use case and a use case realization
  - In other words, this relationship helps separate an interface from its implementation

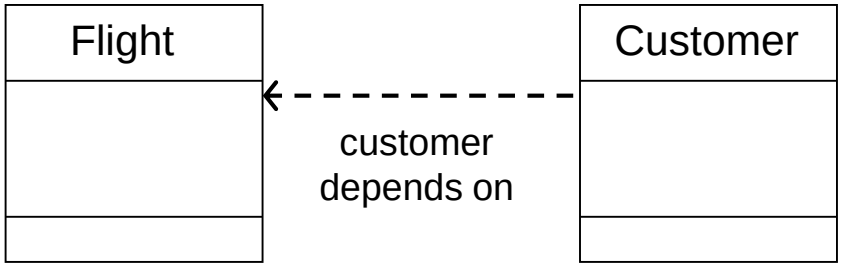
Realizations

- The relationship connects a publicly visible interface (interface class or use case) to the detailed implementation of the interface (class, package, or use case realization)



Dependencies

- A dependency is a semantic relationship where a change to a particular modeling element may affect other modeling elements



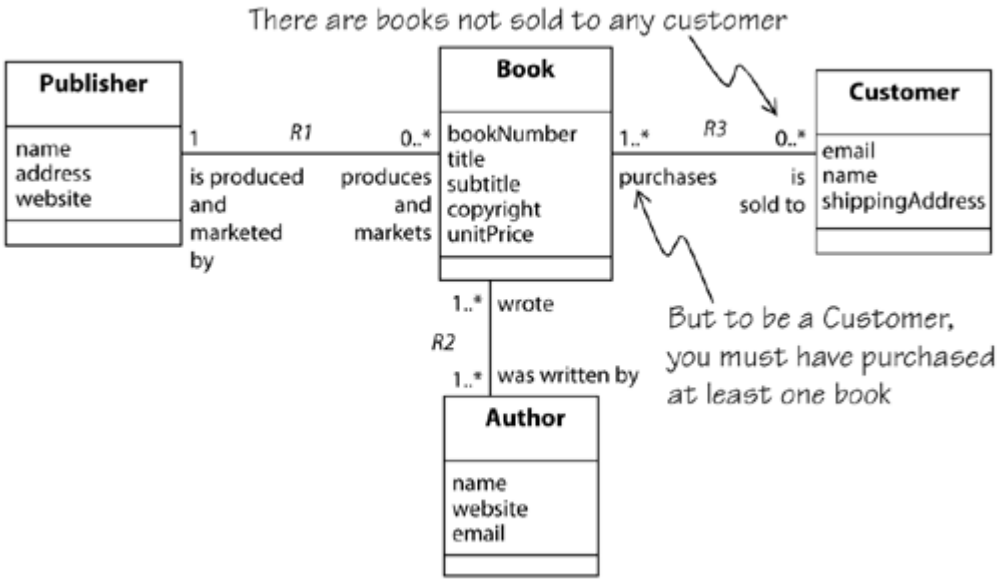
- Dependency describes the need to investigate the model definition of the client element for possible changes if the model definition of the supplier element is changed ('using' relationship, #include in C++)

Multiplicity

- Multiplicity defines how many objects participate in a relationship
  - It is the number of instances of one class related to one instance of the other class
  - For each association and aggregation, there are two multiplicity decisions to make: one for each end of the relationship
  - Multiplicity is represented as a number and a * is used to represent a multiplicity of many

Multiplicity

Multiplicity	Meaning
*	Many
0	Zero
1	One
0..*	Zero or more
1..*	One or more
0..1	Zero or one
1..1	Exactly one



Contents

1. Introduction to UML
2. Structural Diagrams
   - Class Diagram
   - Component Diagram
3. Behavioral Diagrams

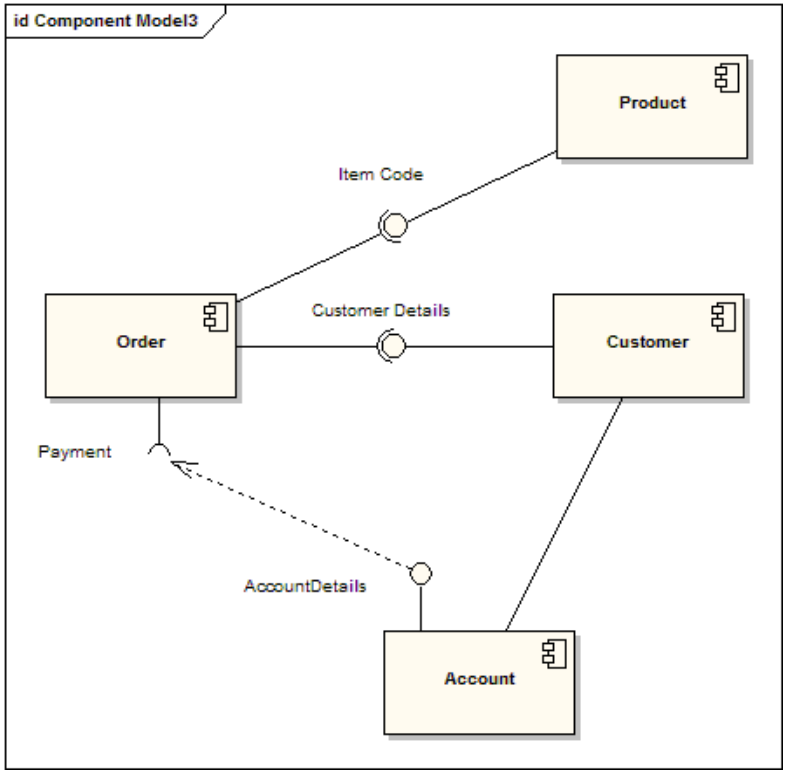
UML Components

- In UML 1.1, a component represented implementation items, such as files and executables
  - Unfortunately, this conflicted with the more common use of the term component," which refers to things such as COM components
  - Over time and across successive releases of UML, the original UML meaning of components was mostly lost

- In UML 2.0, components are considered autonomous, encapsulated units within a system or subsystem that provide one or more interfaces
  - Although the UML 2 specification does not strictly state it, components are larger design units that represent things that will typically be implemented using replaceable modules

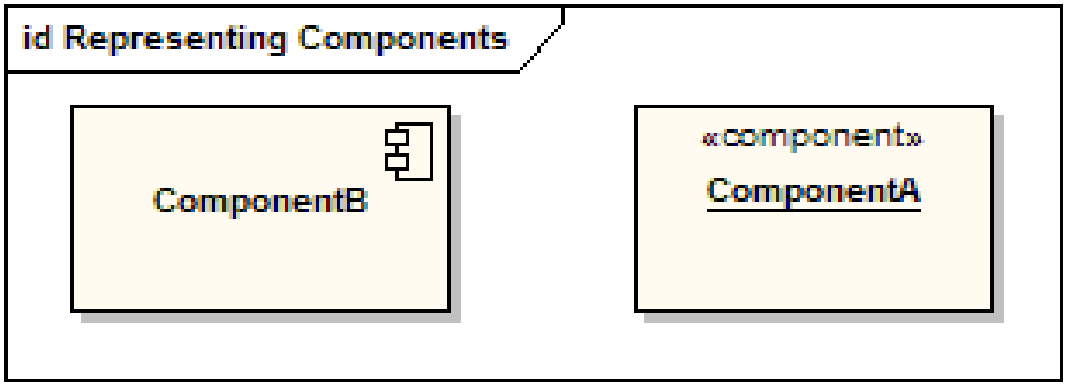
Component Diagram

- A component diagram depicts how a software system is split up into physical components and shows the dependencies among these components
  - Physical components could be files, headers, link libraries, modules, executables, or packages
  - Component diagrams can be used to model and document any system's architecture



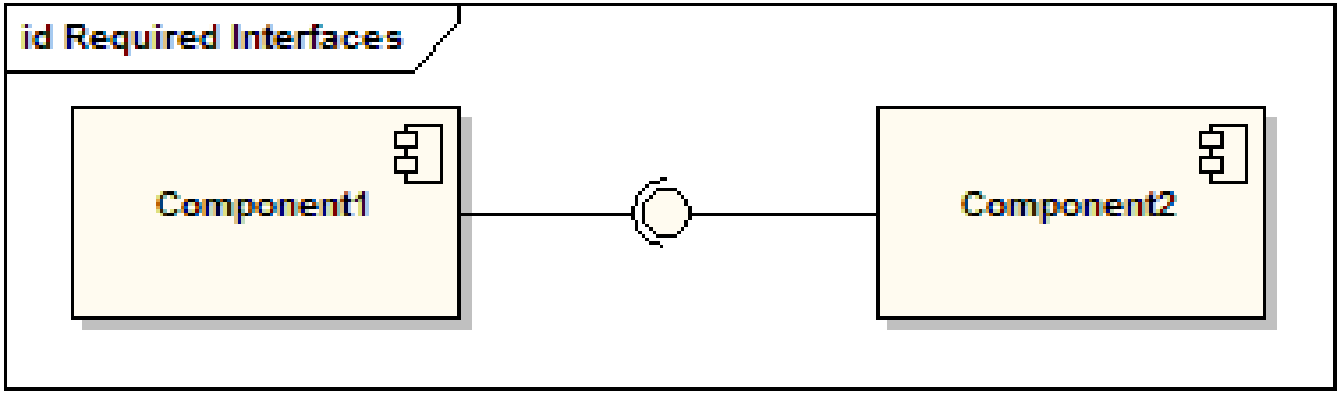
Component Representation

- A component may be displayed as a rectangle with a component icon in the right-hand upper corner
- Components can also be represented as a rectangular classifier with the keyword «component»



Assembly Connector

- The assembly connector bridges a component's required interface (Component1) with the provided interface of another component (Component2); this allows one component to provide the services that another component requires




Q & A